

Table Logic: A Relational Database and Real-Time Seating Tracker

Aaron Graf, Dr. Valerie Ann Haywood, and Aaron Tobias

MSC 5308: Software Engineering

College of Arts and Sciences: Concordia University Texas

for

Dr. Moody Amakobe

Version 1.1

June 7, 2026

**Changelog**

| <b>Version</b> | <b>Date</b>  | <b>Section Changed</b>         | <b>What Changed</b>                                                                                                                                                              | <b>Why</b>                                                                                                                             |
|----------------|--------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>1.0</b>     | May 10, 2026 | All                            | Initial Draft                                                                                                                                                                    | -                                                                                                                                      |
| <b>1.1</b>     | June 5, 2026 | Section 2: Target Users        | Refined roles to include Staff, Customers, and System Admin; matched actors with Use Case Diagram.                                                                               | Discovered during the SRS and UML modeling phases that specialized permissions were needed for administrative and data configurations. |
| <b>1.1</b>     | June 5, 2026 | Section 3: High-Level Features | Formally mapped features to product backlog items (US-001, US-002, US-004, US-005) and added API-driven real-time push synchronization.                                          | Requirements work and sequence modeling clarified the exact endpoints and behaviors required for live seat updates.                    |
| <b>1.1</b>     | June 5, 2026 | Section 4: Technology Stack    | Finalized exact tool versions: React 18.3.x, Vite 5.x, Node.js 20.x, Express 4.x, Sequelize ORM 6.x, and Neon.tech PostgreSQL 15. Added JWT/bcrypt, Docker, Jest, and Supertest. | Formally specified during Week 4 Architecture mapping to minimize context switching and lock down development environments.            |
| <b>1.1</b>     | June 5, 2026 | Section 5: Success Criteria    | Replaced vague milestones with explicit, measurable thresholds directly                                                                                                          | Aligned criteria with NFR-01 (UI response under 200ms) and established                                                                 |

|     |              |                                      |                                                                                                  |                                                                                                 |
|-----|--------------|--------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|     |              |                                      | from the Non-Functional Requirements (NFRs).                                                     | production uptime metrics.                                                                      |
| 1.1 | June 5, 2026 | Section 6: Assumptions & Constraints | Added technical containerization rules (Docker) and formal peer code-review deployment barriers. | Discovered deployment constraints when designing the cloud topology and integration pipelines.  |
| 1.1 | June 5, 2026 | Section 7: Team Norms                | Added code integration and testing standard rules.                                               | Enforced a team standard requiring local validations and passing test suites before PR reviews. |

**Table 1.** *Document Changelog: Chronological matrix detailing structural modifications, feature rescoping, and technology stack refinements between Version 1.0 and Version 1.1 based on completed requirements and architectural design phases.*

### Problem Statement

**Status:** UNCHANGED

- **Content:** Maintains focus on the operational bottlenecks of manual, paper-based seating processes, high failure costs, and the communication gap between front-of-house tracking and backend data optimization.

**Target Users:** [UPDATED]

Refined to ensure complete alignment with your UML Use Case Diagram actors and your SRS User Classes.

- [UPDATED] *Host/Manager (Daily Front-of-House Operator)*: Primary daily user managing active seating grid states and guest flows to optimize table turnover. Technical

## REFINED VISION DOCUMENT

profile: Low-to-moderate.

- *Restaurant Owner (Business Stakeholder)*: Interacts weekly to review automated structural layouts, operational efficiency, and metrics.
- *Restaurant Attendee (End Customer)*: Modeled as an actor interacting with the system boundary to view live availability maps and secure accurate wait times.
- [UPDATED] *System Admin (Technical Administrator)*: Added class to account for users managing server infrastructure, configuration files, and direct PostgreSQL relational database schemas. Technical profile: High.

### **High-Level Features [UPDATED]**

Features have been updated to map directly to specific User Stories (US) in your product backlog.

- Feature 1: *Interactive Table Map* [UPDATED]
  - Description: A visual digital floor plan displaying real-time table statuses using color coding (e.g., Green for Open, Red for Occupied).
  - Backlog Mapping: Directly satisfies US-001 (FR-01, FR-02).
- Feature 2: *Host Control Dashboard* [UPDATED]
  - Description: An interactive interface allowing hosts to toggle table statuses (Vacant to Occupied or Dirty) within two clicks.
  - Backlog Mapping: Directly satisfies US-002 (FR-003, FR-004).
- Feature 3: *Capacity Filtering* [UPDATED]
  - Description: Dynamically filters the active floor layout plan to display specific table capacities (e.g., 2-tops vs 4-tops) while hiding non-matching layout elements.

- Backlog Mapping: Directly satisfies US-004 (FR-006, FR-007).
- Feature 4: *Live Data Persistence & Push Sync* [UPDATED]
  - Description: Immediate backend transaction routing that pushes table updates to all connected front-of-house instances via API to prevent double-seating anomalies.
  - Backlog Mapping: Directly satisfies US-003 and US-005 (FR-11, FR-12, FR-15).

### **Technology Stack** [UPDATED]

Updated to transition from high-level stack assumptions to your exact system engineering specification, as formalized in your Architecture Document.

- *Frontend Layer* [UPDATED]:
  - React (v18.3.x) paired with Vite (v5.x) as the build utility for rapid Hot Module Replacement (HMR) and dynamic modular floor plan rendering.
- *Backend Layer* [UPDATED]:
  - Node.js (v20.x LTS) and Express framework (v4.x) running an unopinionated, minimalist REST API engine.
- *Data Access & Persistence* [UPDATED]:
  - PostgreSQL (v15) database instances hosted via Neon.tech cloud infrastructure. Object-relational translations are abstracted via the Sequelize ORM (v6.x) to track system migrations and enforce strict data schema integrity.
- *Security & Authentication Middleware* [UPDATED]:
  - JSON Web Tokens (JWT via jsonwebtoken v9.x) combined with bcrypt for stateless user session validation, are intercepted at the backend routing tier.
- *Testing & Integration Infrastructure* [UPDATED]:

## REFINED VISION DOCUMENT

- Jest (v29.x) for program unit testing, Supertest (v6.x) for isolated backend API endpoint assertions, and Docker (v24.x) for standardized containerized local/production environment setups.
- *Hosting & CI/CD Pipelines:*
  - Deployed to Render web service instances via continuous deployment Git hooks automated through GitHub Actions.

### **Success Criteria [UPDATED]**

Vague assertions are replaced with measurable, verifiable thresholds tied to your System Non-Functional Requirements (NFRs).

- [UPDATED] *Criterion 1 (UI Responsiveness):* The interactive map layout must process status toggles and render updated color-coded table states within a strict window of 200ms of a host's click, satisfying NFR-01.
- [UPDATED] *Criterion 2 (System Reliability):* The deployed live production web application environment must maintain a verifiable operational uptime threshold of 99.9% during active restaurant operating hours (NFR-04).
- [UPDATED] *Criterion 3 (Data Synchronization):* Data transactions must update the cloud PostgreSQL node synchronously with an API response, displaying fallback notifications and state rollbacks if communication fails (FR-14).
- *Criterion 4 (Technical Reproducibility):* A clean local deployment must be verifiable by cold-starting the software from the repository document steps in an isolated container.

### **Assumptions and Constraints [UPDATED]**

Refined based on deployment constraints discovered during architecture work.

- [UPDATED] *Technical Environment Constraint:* The application software architecture

## REFINED VISION DOCUMENT

must build and run successfully within isolated Linux-based Docker containers to align development machines with remote cloud targets.

- [UPDATED] *Security Configuration Constraint*: Environmental templates (.env.example) must separate infrastructure keys from application repository history. Absolutely zero active production values may be exposed in source control tracking.
- *Financial Constraint*: System components must operate entirely within the Free Tier policies of GitHub and Render.

### **Team Roles and Working Norms [UPDATED]**

Updated to capture your team's concrete pull request guidelines and quality gates.

- *Team Roles*:
  - Aaron T. (Project & DevOps Lead), Aaron G. (Frontend Lead), and Valerie H. (Backend/Database Lead).
- [UPDATED] *Code Quality Norms*: All open Pull Requests (PRs) must possess comprehensive automated test assertions (Jest/Supertest) validating new routes or components before peer evaluation.
- [UPDATED] *Verification Norms*: Pull requests require at least one passing code review and successful completion of the automated CI/CD pipeline via GitHub Actions prior to integration into the target.

Document Version Ledger

| Version | Date      | Author | Changes                                                                 |
|---------|-----------|--------|-------------------------------------------------------------------------|
| 1.0     | 5/10/2026 | Team   | Initial Draft                                                           |
| 1.1     | 6/5/2026  | Team   | Updated after requirements and design work (to be completed in Week 5). |

**Table 2.** *Version Control Matrix: Document Iterations, Authorship, and Chronological Change Log.*

Ethical Use of AI

As a novice in computer science with no industry experience, we used Gemini (AI) and Grammarly to develop and align this Vision Document Refinement (v1.1) with our structural engineering work.

- *Gap Analysis and Changelog Formulation:* We consulted Gemini to compare our initial Week 1 assumptions with later technical requirements and architectural decisions. This analysis helped structure the Document Changelog, detailing changes and justifications across project iterations.
- *Feature and Requirements Backlog Mapping:* Gemini cross-referenced our high-level system features with functional requirements (FRs) and user story IDs (such as US-001, US-002, US-005) in the Software Requirements Specification (SRS). This ensured accurate traceability between the refined vision and our development backlog.
- *Quantifying Success Criteria:* We used Gemini to convert our non-measurable success metrics into objective, verifiable quality thresholds. This enabled us to link project success benchmarks directly to quantitative metrics in our Non-Functional Requirements (NFRs), like UI response times and cloud environment availability.
- *Documentation & Technical Writing Standards:* Grammarly was utilized to audit the



## REFINED VISION DOCUMENT

supporting prose for grammatical precision, technical clarity, and stylistic consistency. All final decisions regarding structural modifications, feature boundaries, and changelog justifications were made exclusively by the authors to ensure that the TableLogic system accurately fulfills the rigorous educational requirements of the MSC 5308 course.

### *References*

- Adroit Market Research. (2023). *Restaurant online ordering system market trends and growth drivers: Strategic analysis of Restolabs, Olo, MenuDrive, Toast POS, and ChowNow*.  
<https://www.adroitmarketresearch.com/industry-reports/restaurant-online-orderingsystem-market>
- Conger, S. (2014). *Hands-On Database* (2nd ed.). Pearson.
- Hernandez, M. J. (2013). *Database design for mere mortals: A hands-on guide to relational*

*database design* (3rd ed.). Addison-Wesley.

Lupasc, A., & Huzum, M.-C. (2020). Designing an information system for managing restaurant orders. *Annals of Dunarea de Jos University. Fascicle I: Economics and Applied Informatics*, 26(2), 194–199.

Microsoft. (n.d.). What is SQL Server? Microsoft Learn. Retrieved from

<https://learn.microsoft.com/enus/sql/sqlserver/what-is-sql-server?view=sql-server-ver17>

Naidu, M. S. S., Pravin, A., Rao, I. R. S. N., & Sivaranjani, K. (2024). Embedded system-based restaurant automation. *International Journal of Emerging Research in Engineering, Science, and Management*, 3(4), 29–32.

<https://doiorg.concordia.idm.oclc.org/10.58482/ijeresm.v3i4.4>

Pymnts. (2024). *Starbucks Uses AI-Powered Personalized Rewards to Boost Frequency and Spend*. Retrieved from

<https://www.pymnts.com/news/loyalty-and-rewardsnews/2024/starbucks-uses-artificial-intelligence-powered-personalized-rewards-boostfrequency-check-size/>

Tech Monitor Staff Writer. (2023, January 6). What is SQL Server? Tech Monitor. Retrieved from <https://www.techmonitor.ai/what-is/what-is-sql-server>

TechTarget. (n.d.). Microsoft SQL Server. SearchDataManagement. Retrieved from

<https://www.techtarget.com/searchdatamanagement/definition/SQL-Server>

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.